Chapter 5. Neural Networks and Backpropagation

Basic concepts of neural networks, multilayer perceptrons, backpropagation

Contents

1 From Hand-Designed Features to Learned Functions **2**

2 Neurons, Layers and Shapes **2**

3 Why Nonlinearity and Depth **2**

4 Activation Functions and Outputs **3**

5 Computation Graphs and the Scalar Chain Rule **3**

6 Vector Calculus Rules for Backpropagation **4**

6.1 Backpropagation through an MLP 5

7 Worked Results **5**

7.1 Single neurons: MSE versus cross-entropy 5

7.2 Residual MLP 5

1 From Hand-Designed Features to Learned Functions

A classifier maps an observation $\mathbf{x}$ to an output $y = f(\mathbf{x}; \boldsymbol{\theta})$. Classical pipelines fix features by hand (color histograms, filter banks) and learn only a final classifier; deep learning trains the representation and the classifier end to end from labeled data. Two properties of features recur: a feature is *invariant* to a transformation if it does not change, and *equivariant* if it changes in the same way (Chapter 7).

2 Neurons, Layers and Shapes

Definition 5.1: multilayer perceptron

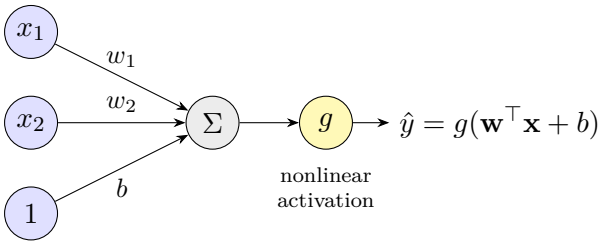
A neuron computes $h = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. A fully connected layer stacks neurons:

$$\mathbf{z}^{(\ell)} = W_\ell \mathbf{a}^{(\ell-1)} + \mathbf{b}_\ell, \qquad \mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)}), \qquad \mathbf{a}^{(0)} = \mathbf{x},$$

with $W_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}_\ell \in \mathbb{R}^{n_\ell}$, and σ applied elementwise. An MLP is a composition of such layers; a 1000-neuron layer is one layer, not 1000 layers.

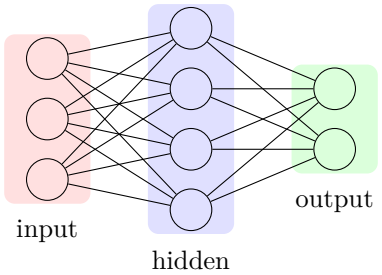
Example 5.2: parameter count

A two-layer classifier $\mathbf{s} = W_2 \,\mathrm{ReLU}(W_1 \mathbf{x})$ on CIFAR-10 images ($32 \times 32 \times 3 = 3072$ inputs), hidden width 100, 10 classes: $W_1 \in \mathbb{R}^{100 \times 3072}$, $W_2 \in \mathbb{R}^{10 \times 100}$, so $307{,}200 + 1000 = 308{,}200$ weights (plus 110 biases).



A single neuron: an affine combination of the inputs followed by a nonlinear activation g.

$$f(\mathbf{x}) = \mathrm{softmax}\left(W_2 \max(0, W_1 \mathbf{x})\right)$$



A two-layer network: every unit of a layer is connected to every unit of the next.

3 Why Nonlinearity and Depth

Without activations layers collapse: $W_2(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2 W_1)\mathbf{x} + (W_2 \mathbf{b}_1 + \mathbf{b}_2)$ is still affine, however many layers there are, and an affine score yields linear decision boundaries. Hidden layers

are learned embeddings: e.g. $\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$ separates an inner from an outer circle by a plane. A network learns such combinations instead of receiving them by hand; depth composes them hierarchically (strokes $\to$ loops $\to$ digits). Depth increases expressiveness but does not by itself guarantee trainability or generalization (Chapters 6–8).

4 Activation Functions and Outputs

Activation	Definition	Derivative
Sigmoid	$\sigma(z) = (1 + e^{-z})^{-1} \in (0, 1)$	$\sigma(1 - \sigma) \in (0, 1/4]$
Tanh	$\tanh z = 2\sigma(2z) - 1 \in (-1, 1)$	$1 - \tanh^2 z \in (0, 1]$
ReLU	$\max(0, z)$	$\mathbf{1}\{z > 0\}$
Leaky ReLU	$\max(az, z)$, $0 < a < 1$	1 or a
ELU	z $(z \geq 0)$, $a(e^z - 1)$ $(z < 0)$	1 or ae^z
Maxout	$\max_k(\mathbf{w}_k^\top \mathbf{x} + b_k)$	routes to the max

Outputs: softmax for mutually exclusive classes (loss $-\log p_y$, score gradient $\mathbf{p} - \mathbf{e}_y$), sigmoid for binary probabilities, identity for regression.

Example 5.3: forward pass

$\mathbf{x} = (1, -2)$, $W_1 = \begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}$, $W_2 = \mathbf{I}$, no biases, ReLU hidden layer: $\mathbf{z}_1 = (-1, 4)$, $\mathbf{a}_1 = (0, 4)$, scores $(0, 4)$, softmax $\approx (0.018, 0.982)$.

5 Computation Graphs and the Scalar Chain Rule

Training repeats: zero the gradients, forward pass, loss, backward pass (backpropagation computes derivatives), optimizer update. Backpropagation applies the chain rule on a computation graph: each node multiplies the *upstream* derivative (from later operations) by its *local* derivative.

Example 5.4: $f = (x + y)z$

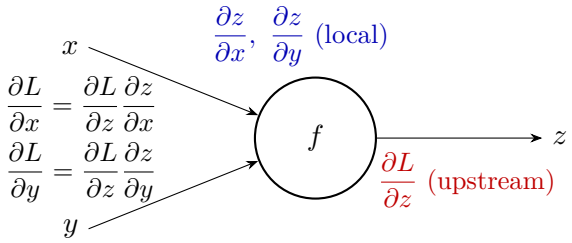
At $(x, y, z) = (-2, 5, -4)$: $q = x + y = 3$, $f = qz = -12$. Local derivatives $\partial f/\partial q = z = -4$, $\partial f/\partial z = q = 3$, $\partial q/\partial x = \partial q/\partial y = 1$, hence $\partial f/\partial x = \partial f/\partial y = -4$, $\partial f/\partial z = 3$.

Example 5.5: sigmoid neuron

$f = \sigma(w_0x_0 + w_1x_1 + w_2)$ with $w_0 = 2, x_0 = -1, w_1 = -3, x_1 = -2, w_2 = -3$: $z = 1$, $f \approx 0.731$, $\partial f/\partial z = f(1 - f) \approx 0.197$. Then $\partial f/\partial w_0 = 0.197x_0 = -0.197$, $\partial f/\partial x_0 = 0.197w_0 = 0.393$, $\partial f/\partial w_1 = -0.393$, $\partial f/\partial x_1 = -0.590$, $\partial f/\partial w_2 = 0.197$. Decomposing σ into $z \to -z \to e^{-z} \to 1 + e^{-z} \to 1/(\cdot)$ gives the same numbers; treating σ as one node is cheaper.

Remark 5.6: gate patterns

Add gate: distributes the upstream derivative to both inputs. Multiply gate: each input receives upstream $\times$ the other input. Copy (a value used twice): derivatives add. Max gate: routes the derivative to the maximal input. Hence a shared parameter accumulates gradients from all its uses.



Backpropagation at one node: the downstream gradient is the upstream gradient times the local gradient.

6 Vector Calculus Rules for Backpropagation

Motivation. Backpropagation is the chain rule organized so that each layer passes one vector backwards. A handful of vector rules (for linear maps, elementwise nonlinearities and losses) suffice to differentiate any feedforward network.

Gradient convention: $\nabla_{\mathbf{x}} L$ has the shape of $\mathbf{x}$; the Jacobian of $\mathbf{y} \in \mathbb{R}^m$ in $\mathbf{x} \in \mathbb{R}^n$ is $m \times n$ with entries $\partial y_i / \partial x_j$.

Theorem 5.7: rules R1–R6

R1. Sum rule: the Jacobian of a sum is the sum of Jacobians; terms not depending on the variable contribute 0.

R2. Identity: $\partial \mathbf{x} / \partial \mathbf{x} = \mathbf{I}$.

R3. Chain rule: $\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ (outer Jacobian on the left).

R4. Affine map: $\partial(A\mathbf{x} + \mathbf{b}) / \partial \mathbf{x} = A$.

R5. Elementwise map: $\partial \sigma(\mathbf{z}) / \partial \mathbf{z} = \operatorname{diag}(\sigma'(\mathbf{z}))$, so $\operatorname{diag}(\sigma'(\mathbf{z}))\mathbf{v} = \sigma'(\mathbf{z}) \odot \mathbf{v}$.

R6. Gradient chain rule (scalar L): $\nabla_{\mathbf{x}} L = \left(\frac{\partial \mathbf{u}}{\partial \mathbf{x}}\right)^{\top} \nabla_{\mathbf{u}} L$.

Proof. R6: by R3 the row vector $\partial L / \partial \mathbf{x} = (\partial L / \partial \mathbf{u})(\partial \mathbf{u} / \partial \mathbf{x})$; transposing reverses the order. ■

Theorem 5.8: affine layer

For $\mathbf{z} = W\mathbf{a} + \mathbf{b}$ and $\boldsymbol{\delta} = \nabla_{\mathbf{z}} L$:

$$\nabla_W L = \boldsymbol{\delta}\, \mathbf{a}^{\top}, \qquad \nabla_{\mathbf{b}} L = \boldsymbol{\delta}, \qquad \nabla_{\mathbf{a}} L = W^{\top} \boldsymbol{\delta}.$$

For an activation layer $\mathbf{h} = \sigma(\mathbf{z})$: $\nabla_{\mathbf{z}} L = \nabla_{\mathbf{h}} L \odot \sigma'(\mathbf{z})$.

Proof. $\nabla_{\mathbf{a}} L$: R6 with R4. $\nabla_{\mathbf{b}} L$: R6 with $\partial \mathbf{z} / \partial \mathbf{b} = \mathbf{I}$ (R1, R2). $\nabla_W L$: row i of W, $\mathbf{w}_i^{\top}$, affects L only through $z_i = \mathbf{w}_i^{\top} \mathbf{a} + b_i$, whose gradient in $\mathbf{w}_i$ is $\mathbf{a}$ (R4); thus $\nabla_{\mathbf{w}_i} L = \delta_i \mathbf{a}$, and stacking rows gives $\boldsymbol{\delta} \mathbf{a}^{\top}$. Equivalently, componentwise $\partial L / \partial W_{ij} = \delta_i a_j$. Activation: R6 with R5, and a diagonal matrix is symmetric. ■

Remark 5.9: differentials and trace

Matrix gradients can also be read from differentials: if $dL = \operatorname{tr}(K\, dW)$ then $\nabla_W L = K^{\top}$. For $\mathbf{z} = W\mathbf{a}$, $dL = \boldsymbol{\delta}^{\top}(dW)\mathbf{a} = \operatorname{tr}(\mathbf{a}\boldsymbol{\delta}^{\top} dW)$, so $\nabla_W L = \boldsymbol{\delta}\mathbf{a}^{\top}$.

6.1 Backpropagation through an MLP

With $\boldsymbol{\delta}_\ell = \nabla_{\mathbf{z}^{(\ell)}} L$ (an $n_\ell \times 1$ column), the recursion is

$$\boldsymbol{\delta}_\ell = (W_{\ell+1}^\top \boldsymbol{\delta}_{\ell+1}) \odot \sigma'(\mathbf{z}^{(\ell)}), \qquad \nabla_{W_\ell} L = \boldsymbol{\delta}_\ell \, \mathbf{a}^{(\ell-1)\top}, \qquad \nabla_{\mathbf{b}_\ell} L = \boldsymbol{\delta}_\ell.$$

It follows from applying R6 twice: through the affine map ($W_{\ell+1}^\top$) and through the activation (elementwise product). In a batch, outer products are summed (or averaged) over samples.

Example 5.10: forward example continued backward

Take class 1 as correct: $\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{e}_1 \approx (-0.982, 0.982)$, $\nabla_{W_2} L = \boldsymbol{\delta}_2 \mathbf{a}_1^\top = \begin{pmatrix} 0 & -3.928 \\ 0 & 3.928 \end{pmatrix}$. The first hidden pre-activation was negative, so $\boldsymbol{\delta}_1 = (0, 0.982)$ and $\nabla_{W_1} L = \boldsymbol{\delta}_1 \mathbf{x}^\top = \begin{pmatrix} 0 & 0 \\ 0.982 & -1.964 \end{pmatrix}$: the inactive unit receives no gradient (a "dead ReLU" for this input). For elementwise ReLU, upstream $(4, -1, 5, 9)$ at $\mathbf{x} = (1, -2, 3, -1)$ becomes $(4, 0, 5, 0)$ with no Jacobian ever formed.

7 Worked Results

Motivation. Two standard computations show what the rules reveal: why cross-entropy trains a sigmoid neuron faster than squared error, and how a residual connection keeps the backward signal from vanishing.

7.1 Single neurons: MSE versus cross-entropy

Theorem 5.11: gradient bounds

For $y = \sigma(wx + b)$ and $C = \frac{1}{2}(y - z)^2$: $\nabla C = y(1-y)(y-z)(x, 1)$ and $\|\nabla C\| < \frac{1}{4}\sqrt{1 + x^2}\,(1 + |z|)$. For cross-entropy $C = -[z \log y + (1 - z) \log(1 - y)]$: $\nabla C = (y - z)(x, 1)$.

Proof. Chain rule with $\sigma' = y(1 - y) \le 1/4$ and $|y - z| \le |y| + |z| < 1 + |z|$ since $0 < y < 1$. For cross-entropy, $\partial C / \partial t = -[z(1 - y) - (1 - z)y] = y - z$: the factor σ' cancels. ∎

For a saturated wrong neuron ($z = 1$, $wx + b = -10$), the MSE gradient carries the factor $\sigma' \approx 4.5 \times 10^{-5}$ while the cross-entropy gradient is of order one: MSE with sigmoid suffers a learning slowdown. GD reads $(w_{n+1}, b_{n+1}) = (w_n, b_n) - \eta \nabla C$.

7.2 Residual MLP

Consider $y_\ell = \sigma(A_\ell y_{\ell-1} + b_\ell) + y_{\ell-1}$ for $2 \le \ell \le L - 1$, $y_1 = \sigma(A_1 x + b_1)$, $y_L = A_L y_{L-1} + b_L$ (scalar). Put $z_\ell = A_\ell y_{\ell-1} + b_\ell$, $D_\ell = \operatorname{diag}(\sigma'(z_\ell))$ and $g_\ell = \partial y_L / \partial y_\ell \in \mathbb{R}^{1 \times m}$.

Theorem 5.12: residual Jacobians and gradients

$$\frac{\partial y_\ell}{\partial y_{\ell-1}} = I + D_\ell A_\ell \ (2 \le \ell \le L - 1), \qquad \frac{\partial y_L}{\partial y_{L-1}} = A_L, \qquad g_\ell = A_L \prod_{j=L-1}^{\ell+1} (I + D_j A_j),$$

$$\frac{\partial y_L}{\partial b_\ell} = g_\ell D_\ell, \qquad \frac{\partial y_L}{\partial A_\ell} = D_\ell g_\ell^\top y_{\ell-1}^\top \quad \Big(\tfrac{\partial y_L}{\partial (A_\ell)_{ij}} = (g_\ell)_i \sigma'((z_\ell)_i)(y_{\ell-1})_j\Big),$$

and $\partial y_L/\partial b_L = 1$, $\partial y_L/\partial A_L = y_{L-1}^\top$.

Proof. R1 splits y_ℓ into $\sigma(z_\ell)$ and $y_{\ell-1}$; R2 gives I; R3 with R5, R4 gives $D_\ell A_\ell$. For b_ℓ, the residual term does not depend on b_ℓ and $\partial z_\ell/\partial b_\ell = I$, so $\partial y_\ell/\partial b_\ell = D_\ell$; chain with g_ℓ. For A_ℓ, the ith row affects y_L only through $(z_\ell)_i$, giving the componentwise formula; it factorizes into an i-part and a j-part, i.e. an outer product. ■

If $A_j = 0$ or $\sigma'(z_j) = 0$ for some intermediate j, the factor is $I + 0 = I$, not 0: the identity path carries the gradient, so lower-layer gradients need not vanish. Without the residual, the factor is $D_j A_j = 0$ and all lower gradients are exactly zero. Expanding $\prod(I + D_j A_j) = I + \sum D_j A_j + \cdots$ exhibits the skip path explicitly (Chapter 7).